

Quand c'est presque gratuit, c'est quand même toi le produit

H. B.

Une smartwatch à 8€ livrée avec un collecteur de données personnelles.

Date d'analyse : Juin 2026 - Juillet 2026

Cet article présente les résultats d'une investigation forensique conduite à des fins de recherche en sécurité, dans le respect du cadre légal applicable.

Résumé

Une smartwatch vendue huit euros sur les grandes marketplaces mondiales est livrée avec une application compagne (HeroBand III, identifiée `com.crrepa.band.hero`) qui se révèle être, à l'analyse statique, un collecteur de données personnelles. Quatre flux d'exfiltration opèrent simultanément et indépendamment : fingerprinting matériel profond (IMEI, IMSI, adresse MAC) via le SDK Artillery, tracking comportemental en temps réel via des SDKs d'analytics cloud, surveillance audio dont le flux est traité par le service ASR de Baidu, et accès aux données personnelles (SMS, journal d'appels, contacts). L'objet physique n'est que le prétexte : l'objectif réel est l'acquisition d'un accès permanent et silencieux aux données contenues dans le téléphone de l'acheteur. Le seul responsable identifié est la société **Moyoung/CRREPA**, qui a délibérément assemblé ces outils.

Table des matières

1	Introduction	2
2	L'acteur : Moyoung / CRREPA	2
3	L'écosystème de collecte	3
3.1	L'identité matérielle du téléphone	3
3.2	L'UTDID : l'identifiant qui survit à la désinstallation	4
3.3	Le schéma de collecte comportementale	5
3.4	L'accès aux données personnelles	6
4	La chaîne de monétisation des données	6
5	L'infrastructure d'exfiltration	8
6	La surveillance audio	8
7	Chiffrement de la configuration	9
8	Indicateurs de compromission (IoC) et attribution	10
	Références	11

1 Introduction

Le marché des objets connectés bon marché, dominé par des fabricants chinois dont les produits sont distribués mondialement via AliExpress et Amazon, constitue un vecteur d'entrée massif pour la collecte de données personnelles. Ces appareils (montres connectées, trackers fitness, écouteurs, etc) requièrent souvent une application compagne sur le téléphone de l'utilisateur, lui-même bien plus riche en données que l'objet physique. L'application HeroBand III, distribuée gratuitement en accompagnement d'une smartwatch vendue moins de dix euros, a été soumise à une analyse statique. L'analyse présente dans la suite de ce document présente les conclusions concernant cette application spécifique.

2 L'acteur : Moyoung / CRREPA

L'application est signée par **Moyoung (Shenzhen)**, dont le certificat auto-signé porte la mention CN=moyoung, L=shenzhen, ST=shenzhen, C=cn.

TABLE 1 – Certificat de signature de l'APK.

Champ	Valeur
Sujet	CN=moyoung, L=shenzhen, C=cn
Émetteur	CN=moyoung (auto-signé)
Émis le	2017-12-06
Expire le	2042-11-30 (25 ans)
Algorithme	SHA256withRSA, 2048 bits
SHA256	37:AE:1A:B5:53:08:19:42:0A:C6:14:9D:66:EC:B0:16...

Le certificat auto-signé de vingt-cinq ans garantit que toutes les mises à jour futures de l'application (quel que soit leur contenu) seront acceptées silencieusement par les appareils sur lesquels elle est déjà installée.

L'application n'est pas liée à un seul modèle de montre. Il est montré dans la suite de cette analyse que la base de données déchiffrée depuis l'APK révèle **242 modèles référencés** (99 actifs, 143 supprimés), commercialisés sous des dizaines de marques différentes (SKG, MPRO, SmartBand, BYS, FIT, BR, KW...). Moyoung est une plateforme OEM et vend la même infrastructure logicielle sous des identités multiples, HeroBand III étant le dénominateur commun et donc le point d'entrée vers des millions de téléphones.

Architecture à deux identités

Le code source révèle deux entités distinctes : **CRREPA** (`com.crrepa.band`) pour le package principal et les serveurs `api.crrepa.com`, et **Moyoung** (`com.moyoung.dafit`) pour la configuration réseau et les endpoints `api.moyoung.com`. Cette architecture suggère soit une relation fournisseur/intégrateur, soit une stratégie de dissémination de la responsabilité.

Le code source décompilé présente un taux d'obfuscation de **97%**. En effet, 375 packages sur 386 sont renommés en combinaisons "illisibles" (`a`, `b0`, `z3`...). Les seuls packages lisibles sont ceux des SDKs tiers qui imposent leurs noms pour fonctionner (Alibaba, Microsoft, Tencent). L'intégralité du code propriétaire de l'application est opaque à l'analyse. Une obfuscation de ce type dans une application grand public est un indicateur de volonté de dissimulation (ce qui peut se comprendre pour des produits de qualités supérieures l'est moins pour un produit si bas de gamme). Pour une montre quasi-gratuite, il est étonnant de voir autant d'investissement sur l'obfuscation du code.

3 L'écosystème de collecte

L'application embarque quatre SDKs de collecte de données opérant simultanément et indépendamment, chacun avec ses propres pipelines d'exfiltration :

TABLE 2 – SDKs de collecte embarqués.

SDK	Éditeur	Données collectées
<code>com.alibaba.mtl</code>	Alibaba	Comportement in-app, IMEI, IMSI, UTDID
<code>com.alibaba.sdk.android.beacon</code>	Alibaba	Tracking page par page
<code>com.ta.utdid2</code>	Alibaba / Taobao	Fingerprint persistant cross-app
<code>com.artillery.ctc</code>	Artillery (inconnu)	MAC, Android ID, root, IP
<code>com.tencent.bugly</code>	Tencent	Crashes (stack traces avec données)
<code>com.microsoft.cognitiveservices</code>	Microsoft	Pipeline audio KWS

Les SDKs de collecte dans cette application destinée à une montre connectée à 8€ ont probablement une explication commerciale.

3.1 L'identité matérielle du téléphone

Dès le premier lancement, l'application construit un identifiant composite et persistant de l'appareil. L'IMEI et l'IMSI sont collectés via le SDK Alibaba Tbreast et mis en cache statique :

```

1 // com/alibaba/sdk/android/tbreast/utils/DeviceUtils.java
2 public static String getImei(Context context) {
3     String uniqueID = getUniqueID();
4     imei = uniqueID; // cache statique -- collecte une fois,
5                       conserve indefiniment
6     return uniqueID;
7 }

```

Listing 1 – Collecte IMEI/IMSI: DeviceUtils.java (SDK Alibaba)

Le SDK Artillery ajoute une troisième couche avec une récupération de l'adresse MAC par **trois méthodes en cascade**, conçue pour contourner les protections progressivement introduites par Android :

```

1 // com/artillery/ctc/uitls/DeviceUtils.java
2
3 // Methode 1 -- WifiManager (bloquee depuis Android 10)
4 WifiManager wm = context.getSystemService("wifi");
5 return wm.getConnectionInfo().getMacAddress();
6
7 // Methode 2 -- NetworkInterface via InetAddress (fallback)
8 NetworkInterface ni = NetworkInterface.getByInetAddress(
9     getInetAddress());
10 return formatMAC(ni.getHardwareAddress());
11
12 // Methode 3 -- Scan direct de l'interface wlan0 (dernier recours)
13
14 for (NetworkInterface ni : NetworkInterface.getNetworkInterfaces())
15     if (ni.getName().equalsIgnoreCase("wlan0"))
16         return formatMAC(ni.getHardwareAddress());

```

Listing 2 – Récupération MAC en cascade: DeviceUtils.java (SDK Artillery)

Au total, l'identité matérielle collectée comprend : IMEI, IMSI, adresse MAC (WiFi), Android ID, modèle, marque, résolution d'écran, opérateur téléphonique, type de réseau, version Android, architectures CPU.

3.2 L'UTDID : l'identifiant qui survit à la désinstallation

L'UTDID (Universal Taobao Device ID) est l'identifiant persistant d'Alibaba. Contrairement à l'Android ID réinitialisé à la désinstallation, l'UTDID est conçu pour **survivre aux désinstallations** en exploitant plusieurs emplacements de stockage. Il est injecté dans chaque requête Beacon envoyée à Alibaba Cloud :

```

1 // com/alibaba/sdk/android/beacon/c.java
2 return new b.a()
3     .f(UTDevice.getUtdid(context)) // UTDID injecte dans chaque
      requete
4     .a(map)
5     .a();
6 // Destination : beacon-api.aliyuncs.com

```

Listing 3 – Injection UTDID dans chaque payload Beacon

L'UTDID est **cross-applications**, ce qui signifie que tous les éditeurs qui intègrent un SDK Alibaba partagent le même espace d'identifiants UTDID. Par conception, cet identifiant pourrait permettre à Moyoung (ou à tout tiers ayant accès aux données collectées) de corréler le comportement d'un même utilisateur entre plusieurs applications intégrant les mêmes SDKs. Il est important de noter que cette corrélation n'est pas une action d'Alibaba, c'est bien Moyoung qui a délibérément choisi d'intégrer cette propriété dans son application.

3.3 Le schéma de collecte comportementale

Le SDK Alibaba MTL Log est un outil d'analytics cloud public, comparable à Google Analytics ou Mixpanel, et définit un schéma exhaustif de champs collectés qui sont remontés vers les serveurs analytics de Moyoung via l'infrastructure Alibaba :

```

1 // com/alibaba/mtl/log/model/LogField.java
2 public enum LogField {
3     IMEI, IMSI, BRAND, DEVICE_MODEL, RESOLUTION,
4     CARRIER, ACCESS, ACCESS_SUBTYPE, // operateur + type
      reseau
5     CHANNEL, APPKEY, APPVERSION,
6     LL_USERNICK, USERNICK, LL_USERID, USERID,
7     LANGUAGE, OS, OSVERSION, SDKVERSION,
8     START_SESSION_TIMESTAMP, UTDID,
9     RESERVE2, RESERVE3, RESERVE4, RESERVE5, RESERVES,
10    RECORD_TIMESTAMP,
11    PAGE, // chaque ecran
      consulte

```

```

12     EVENTID , ARG1 , ARG2 , ARG3 , ARGS           // chaque interaction
13 }

```

Listing 4 – Schéma complet de collecte : LogField.java (SDK Alibaba MTL)

Ces données sont accumulées localement et exfiltrées par un `UploadEngine` en service de fond, avec un intervalle adaptatif (plus court en foreground pour ne pas manquer d'événements, plus long en background pour préserver la batterie et minimiser la détection).

3.4 L'accès aux données personnelles

L'application demande l'accès aux SMS, au journal d'appels et aux contacts, présentés comme nécessaires pour afficher les notifications sur la montre. Cette justification est fonctionnellement légitime. Une smart watch est essentiellement faite pour présenter ces fonctionnalités à l'utilisateur. En revanche, il faut savoir qu'Android n'isole pas le code de l'application principale des SDKs tiers embarqués. Ainsi, une fois la permission `READ_SMS` accordée, **les SDKs Alibaba et Artillery opèrent dans le même processus** et accèdent techniquement aux mêmes ressources.

La permission `ACCESS_FINE_LOCATION`, présentée pour la météo localisée, transmet les coordonnées GPS exactes à chaque consultation :

```

1 // DeviceApiStores.java
2 @GET("https://wr.moyoung.com/v2/wr-7")
3 Observable<WeatherForecast> getWeather(
4     @Query("lat") double latitude,
5     @Query("lon") double longitude
6 );

```

Listing 5 – Transmission GPS silencieuse à chaque requête météo

Ainsi, chaque fois que l'utilisateur consulte la météo sur sa montre, sa position exacte est transmise à un serveur chinois appartenant à Moyoung.

4 La chaîne de monétisation des données

Le profil publicitaire

La première destination des données est le marché publicitaire de précision. La combinaison IMEI + UTDID + comportement in-app constitue un profil d'une granularité exceptionnelle. L'UTDID permet de relier ce profil à l'historique d'achat sur Taobao et AliExpress, aux recherches effectuées, aux publicités cliquées (l'ensemble de l'empreinte numérique de l'utilisateur dans l'écosystème Alibaba).

Ce profil est exploité sur les Data Management Platforms (DMP) chinoises. D'après L'IA d'Anthropic Claude Sonnet, sa valeur marchande peut être de quelques centimes par unité. Cette valeur est à titre indicative et reste à confirmer. Si validé, on peut estimer que sur une base de plusieurs millions d'utilisateurs, le modèle économique sur une base mensuelle ou hebdomadaire semble parfaitement viable. Le prix de vente de la montre de 8€ permet *a minima* une marge de 50%. En estimant une vente des données à 0,4€ mensuel, on peut prédire que la rentabilité du dispositif s'effectue en moins d'un an.

La base de données téléphoniques géolocalisée

La collecte systématique de l'opérateur, du type de réseau et des coordonnées GPS permet de constituer une base de données d'abonnés géolocalisés. Ce type de base a une valeur commerciale directe :

- **Ciblage publicitaire contextuel** : segmentation par opérateur et localisation.
- **Fraude téléphonique ciblée** : bases d'opérateurs et numéros directement exploitables pour le smishing et le vishing à grande échelle.
- **Revente d'abonnements premium** : numéros associés à des profils comportementaux utilisés pour des arnaques aux services surtaxés.

L'audio traité par un service ASR externe

Pour la reconnaissance vocale, Moyoung a choisi d'intégrer l'API DuerOS de Baidu. Il s'agit d'un service cloud de reconnaissance automatique de la parole (ASR), comparable à AWS Transcribe ou Google Speech-to-Text. L'audio capturé est transmis à l'infrastructure Baidu pour transcription ; la réponse est ensuite renvoyée à l'application.

Données confiées à une infrastructure soumise à la loi 2017

Tout prestataire cloud hébergé en Chine est soumis à la loi sur le renseignement national de 2017, qui contraint les entreprises à coopérer avec les services de renseignement. **Moyoung** a fait le choix de confier des données audio à un prestataire relevant de cette juridiction, sans en informer l'utilisateur ni lui proposer d'alternative.

La fraud analytics

Le SDK Artillery collecte également le statut root, la présence d'un émulateur, et l'état du débogage ADB. Ces données permettent de distinguer un vrai utilisateur d'un environnement de test ou d'analyse. Dans le secteur de la fraude publicitaire, détecter les fermes de clics versus les vrais utilisateurs est une donnée qui se vend. Elles permettent également à l'application d'adapter son comportement selon qu'elle est ou non sous surveillance.

5 L'infrastructure d'exfiltration

Comme cela a pu être présenté en début d'article, l'application s'appuie sur une infrastructure répartie entre trois opérateurs distincts, chacun gérant ses propres flux de données. Ceux-ci sont présentés en détails ci-dessous :

TABLE 3 – Infrastructure réseau d'exfiltration.

Endpoint	Contenu transmis	Opérateur
beacon-api[.]aliyuncs[.]com	Comportement, UTDID, device ID	Moyoung (via SDK analytics Alibaba)
api[.]moyoung[.]com	Firmware, MAC, watch faces	Moyoung
api2[.]crrepa[.]com	Instructions, config	CRREPA
wr[.]moyoung[.]com	Coordonnées GPS + météo	Moyoung
qcdn[.]moyoung[.]com	CDN firmwares et assets	Moyoung
100[.]67[.]64[.]54	Fallback Beacon (IP hardcodée)	Moyoung (via SDK Alibaba)
API DuerOS ASR	Flux audio PCM 16 kHz	Moyoung (via service ASR Baidu)
SDK Bugly	Crashes, stack traces	Moyoung (via SDK crash Tencent)

Le trafic Alibaba Beacon bascule sur une IP hardcodée (100.67.64.54) en cas d'indisponibilité DNS, rendant le blocage par filtrage DNS **insuffisant**.

6 La surveillance audio

L'élément le plus préoccupant et déjà énoncé dans la section 4 est l'existence d'un pipeline d'écoute continue du microphone. Ce pipeline fonctionne en deux étages. En local, le moteur Microsoft Azure Keyword Spotting (KWS) analyse en continu le flux audio du microphone via un buffer circulaire dédié (`libcbuf.so`). Un modèle de réseau de neurones de 1,9 MB (format ONNX Runtime, embarqué dans la bibliothèque native `libMicrosoft.CognitiveServices.Speech.extension.kws.ort.so`) tourne en permanence sur le CPU du téléphone, sans connexion réseau.

```

1 keyword_spotter_open
2 keyword_spotter_initialize
3 keyword_spotter_write           ; ecriture continue du flux audio
4 FireKeywordDetectedEvent       ; callback : mot de reveil detecte
5 kws-everything-audio-          ; prefixe : capture de tout l'
  audio

```

```
6 session.use_ort_model_bytes_directly
```

Listing 6 – Fonctions du moteur KWS — libMicrosoft...kws.so

Dès que le mot de réveil est détecté, l'étage cloud s'active. L'audio PCM 16 kHz est alors streamé vers les serveurs de Baidu via le protocole **DuerOS** (plateforme IoT intelligente de Baidu).

Le modèle neural embarqué décide de ce qui constitue un « mot de réveil ». L'utilisateur ne contrôle pas ce modèle. Moyoung peut le mettre à jour via le mécanisme OTA (`api[.]moyoung[.]com/v2/upgrade`) sans notification. Le préfixe `kws-everything-audio-` observé dans les strings binaires suggère que le système est capable de capturer de larges fenêtres audio, au-delà du seul fragment immédiatement suivant le mot de réveil.

Pour un utilisateur sensible comme un fonctionnaire, un dirigeant d'entreprise, ou encore un journaliste, cette surface d'exposition est inacceptable. Le moteur s'exécute en permanence, y compris lors de conversations privées, dans des réunions, dans des espaces confidentiels, etc.

7 Chiffrement de la configuration

L'application embarque un fichier de configuration (`assets/config.txt`, 96 832 bytes encodés en Base64) chiffré en Triple-DES. L'effort mis en place pour chiffrer ce fichier (clé fragmentée en deux emplacements distincts dans le binaire et utilisation d'une date comme vecteur d'initialisation) est en soi un signal indiquant que l'éditeur cherche évidemment à dissimuler son contenu en utilisant des techniques déjà observées dans des analyses de programmes malveillants.

La clé est reconstituée à l'exécution depuis deux sources :

```
1 // com/moyoung/dafit/module/common/network/e.java
2 public static SecretKey b() {
3     StringBuilder sb = new StringBuilder();
4     // Partie 1 : depuis resources.arsc (ID 0x7F1301BC)
5     sb.append(context.getString(R.string.des_key)); // valeur :
6     "CB"
7     // Partie 2 : hardcodee dans le code (ASCII 67 a 88)
8     for (int i = 67; i <= 88; i++) {
9         sb.append((char) i); // "CDEFGHIJKLMNOPQRSTUVWXYZ"
10    }
11    return new SecretKeySpec(sb.toString().getBytes(UTF_8), "
        DESede");
12    // Cle complete : "CBCDEFGHIJKLMNOPQRSTUVWXYZ" (24 bytes)
```

```

12 }
13
14 public static IvParameterSpec a() {
15     // IV depuis resources.arsc (ID 0x7F1301BB)
16     return new IvParameterSpec(
17         context.getString(R.string.des_iv).getBytes(UTF_8)
18     ); // valeur : "20160808" -- date de lancement de l'
        application
19 }

```

Listing 7 – Reconstruction de la clé 3DES: SecretKeyManager.java

Ce chiffrement est reconstituable par analyse statique seule. La clé complète (CBCDEFGHIJKL MNOPQRSTUVWXYZ) et l'IV (20160808) sont dérivables sans exécuter l'application. On ne parle donc pas de sécurité cryptographique ici, à moins de discuter de sécurité par obscurantisme.

Le déchiffrement révèle alors un fichier JSON de 72 Ko. Il s'agit de la base de données complète de compatibilité matérielle de la plateforme Moyoung.

TABLE 4 – Contenu de `assets/config.txt` déchiffré.

Champ	Valeur
Modèles actifs	99
Modèles supprimés	143
CDN firmwares	https://qcdn.moyoung.com/
Puces Nordic NRF	66 modèles
Puces Realtek	32 modèles
Monitoring tension artérielle	90 / 99 modèles
Oxymétrie SpO2	94 / 99 modèles
Modèles HID (clavier/souris BLE)	4 (BYS7, B40X, BIM, FIT812)

La décision de chiffrer cette base (dont le contenu est finalement bénin) révèle la priorité de protéger l'URL du CDN de distribution des firmwares ainsi que la liste des partenaires OEM des regards extérieurs. L'opacité est ici au service de la stratégie commerciale, et non de la sécurité des utilisateurs.

8 Indicateurs de compromission (IoC) et attribution

TABLE 5 – Indicateurs réseau.

Indicateur	Type	Contexte
api[.]moyoung[.]com	Domaine	Firmware, compatibilité
api2[.]crrepa[.]com	Domaine	Instructions (HTTP non chiffré)
wr[.]moyoung[.]com	Domaine	Météo + coordonnées GPS
qcdn[.]moyoung[.]com	CDN	Firmwares et assets
pollux[.]moyoung[.]com	Domaine	Liaison IoT montre/compte
beacon-api[.]aliyuncs[.]com	Domaine	Tracking Alibaba Beacon
100[.]67[.]64[.]54	IP	Fallback Beacon (résistant au DNS)

Références

- [1] Alibaba Mobile Security, *UTDID SDK Documentation*, Alibaba Open Platform, 2021.
- [2] Baidu AI Cloud, *DuerOS Device Interface Protocol*, Baidu Developer Documentation, 2022.
- [3] People's Republic of China, *National Intelligence Law*, Article 7, 2017. <https://www.reuters.com/article/us-china-security-lawmaking>
- [4] Microsoft, *Azure Cognitive Services Speech SDK — Keyword Recognition*, Microsoft Documentation, 2023.